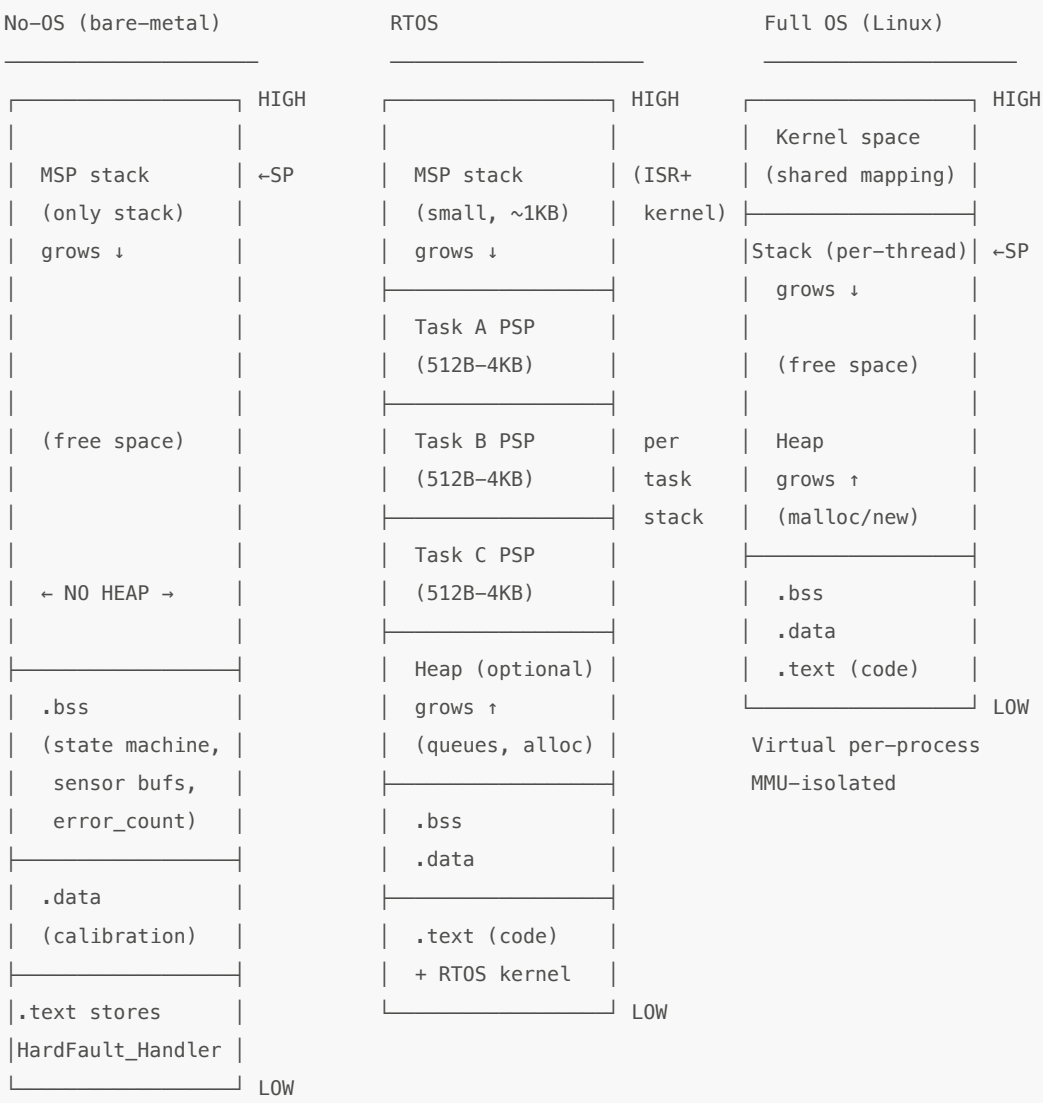
Embedded System Hardware

0. Memory Layout



Key differences

No-OS 1 stack (MSP), no heap, simple layout

- dies if MSP corrupts RTOS 1 MSP + N PSPs, optional heap, stacks adjacent in RAM Full OS virtual memory, per-process isolation, large heap

virtual memory and smart pointer

[Q] compare vector a(10, 0) and int[10] a. in terms of heap / stack / virtual memory structure

Answer: This is a fundamental C++ memory question.

Aspect	`int a[10];` (in a function)	`std::vector<int> a(10, 0);`
Data Location	**Stack**	**Heap**
Control Object	None (the variable `a` is the array)	A small object on the **Stack** that manages the heap data
Allocation	Very fast, single stack pointer adjustment.	Slower, involves a call to the heap allocator (`malloc`).
Lifetime	Automatically destroyed when function exits.	Heap data is automatically freed when the vector object on the stack goes out of scope (RAII).
Flexibility	Size must be a compile-time constant.	Size can be determined and changed at runtime (e.g., `resize()`, `push_back()`).

Heap / Stack issue

`int a[10];`

- puts the data directly on the stack. `std::vector`
- smart containers puts a small manager object on the stack
- and this manager allocates and controls a block of data on the heap. `memory leak` happens if direct memalloc to heap without a proper gc -> `free` has to be explicitly called

MSP Main Stack Pointer vs PSP Process Stack Pointer (dual stack pointers)

- stack pointer points to current top of the stack
- moves at every push/pop including function calls and variables
- fn call grow down the stack pointer while fn returns shrink SP back up

Memory guards

- **No-OS** — MPU region (if configured), 32–256 bytes → protects .bss from MSP overflow
- **RTOS**
    - MPU region, 32 bytes → protects task stacks from MSP overflow
- **Full OS** — Unmapped MMU page, 4 KB → protects adjacent memory from stack overflow — Unmapped MMU page, 4 KB → between heap allocations (ASAN)

---

1. Compute — CPU & Registers

ARM Cortex-M Register Set

Register	Width	Role
r0-r3	32-bit	Function arguments / return value / caller-saved
r4-r11	32-bit	General purpose / callee-saved
r12 (IP)	32-bit	Intra-procedure scratch
SP (r13)	32-bit	Stack pointer (MSP or PSP)

Register	Width	Role
LR (r14)	32-bit	Return address (or magic EXC_RETURN in ISR)
PC (r15)	32-bit	Program Counter — next instruction to execute
xPSR	32-bit	Flags (N/Z/C/V), exception number, thumb bit
CONTROL	2-bit	Privilege level (0=privileged) + stack select (MSP/PSP)

Pipeline: Fetch → Decode → Execute

Fetch : fetch the instruction 0xF1000A05 using Flash address 0x08001000 Decode : translate 0xF1000A05 into real HW actions using CPU native logic Execute : register, ALU executes

```
Cortex-M4: 3-stage pipeline -> improve over sequential

Cycle:    1    2    3    4    5    6
Fetch:   [A]  [B]  [C]  [D]  [E]  [F]
Decode:       [A]  [B]  [C]  [D]  [E]
Execute:           [A]  [B]  [C]  [D]

Branch taken at C:
Cycle:    1    2    3    4    5    6    7
Fetch:   [A]  [B]  [C]  [X]  [X]  [T]  [U]  ← pipeline flush, refill
Decode:       [A]  [B]  [C]  [--] [--] [T]
Execute:           [A]  [B]  [C]       [--]  [T]

Branch = 2-3 cycle penalty (pipeline flush + refill)
```

- This is why tight loops >> frequent branching on MCU
- use predictable branching (for-loop) in MCU

Bit-banding for atomic read-write

- bit-band region maps each BIT to a unique 32-bit ADDRESS
- Writing 1 to that address sets ONLY that bit, atomically

DPC (Direct Peripheral Control) through memory mapping

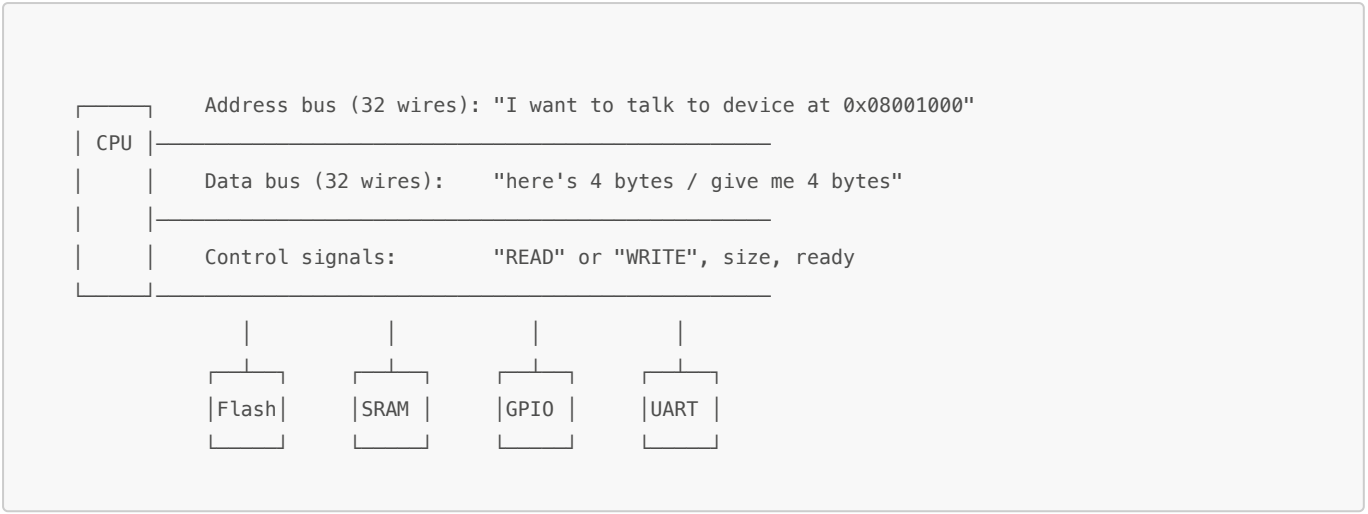
start Peripherals via memory addresses w/o polling

0x00000000-0x07FFFFFF | Flash (code + const data) -> non-volatile 0x20000000-0x3FFFFFFF | SRAM (variables, stack, heap) 0x40000000-0x5FFFFFFF | Peripherals (GPIO, UART, SPI, TIM, ADC...) 0xE0000000-0xFFFFFFFF | System (NVIC, SysTick, SCB, MPU, debug)

volatile = compiler must read/write every time (no caching in register) reinterpret_cast = integer → pointer (different types, same bit pattern)

MCU internal Bus Architecture (AHB / APB) as the pathway for DPC

Bus shares wires that carry address + data between CPU and MCU peripherals



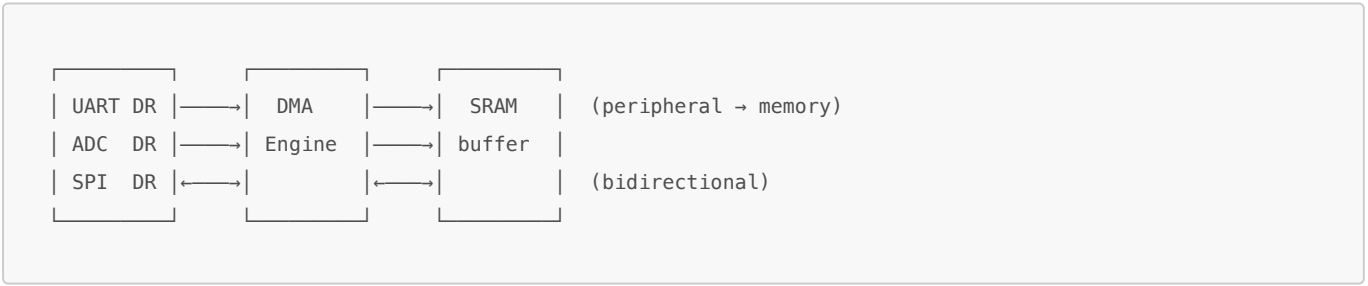
Bus matrix (a hardware router built into the chip) looks at the top bits of the address Based on those bits, one chip-select line (HSEL) electric signal is sent to the device device responds on the data bus

Fetch stage in full:

1. PC = 0x08001000
2. CPU puts 0x08001000 on the AHB address bus → "READ"
3. Flash received HSEL signal outputs 0xF1000A05 on data bus
4. CPU receives 0xF1000A05 → passes to Decode stage
5. PC += 4 (ready for next fetch)

DMA (Direct Memory Access)

CPU configures DMA, then DMA moves data and polling for CPU



DMA modes:

- Peripheral -> Memory: read peripheral data register into RAM buffer. Semi use: UART RX, ADC sampling.
- Memory -> Peripheral: write RAM buffer to peripheral. Semi use: UART TX, DAC output.
- Memory -> Memory: copy between RAM regions. Semi use: buffer management.
- Double-buffer: alternate between two buffers. Semi use: zero-copy processing.

Clock Tree

HSE/HSI → PLL → SYSCLK → AHB (168MHz) for CPU core / SRAM / DMA / GPIO / Flash mem → APB2 (84MHz) for fast peripherals (SPI1, USART1) → APB1 (42MHz) for slow peripherals (USART2, I2C, SPI2)

Wrong clock setup

- UART sends garbage → baud rate calculated from wrong SYSCLK

- SPI too fast for slave → clock divider assumes wrong input frequency

Fault Handlers

- **HardFault** - catch-all for unhandled faults
  - Typical bug: wild pointer points to garbage addr, stack overflow
- **MemManage** — MPU violation hits mem guard
  - Typical bug: task accessing another task's memory
- **BusFault** — invalid bus access
  - Typical bug: reading from disabled peripheral (clock gate off) Each peripheral has a clock gate bit in RCC registers needed enaablement RCC->APB1ENR |= RCC_APB1ENR_USART2EN; // turn on USART2 clock
- **UsageFault** — undefined instruction, unaligned access
  - Typical bug: corrupted function pointer, wrong Thumb bit

CFSR is a 32-bit register split into 3 sub-registers for cpu to know the fault

- Bits [7:0] = MMFSR (MemManage faults — which MPU violation)
- Bits [15:8] = BFSR (BusFault — which bus error)
- Bits [31:16] = UFSR (UsageFault — undefined instr, alignment, etc.) SCB = System Control Block — a set of memory-mapped registers at 0xE000ED00 that control core CPU behavior

```
// Useful HardFault handler — reads stacked registers to find the culprit
void HardFault_Handler() {
    __asm volatile (
        "tst lr, #4      \n"  // check whether MSP or PSP is used
        "ite eq           \n"
        "mrseq r0, msp   \n"  // MSP if in handler mode
        "mrsne r0, psp   \n"  // PSP if in thread mode
        "b fault_dump     \n"  // pass stack frame pointer to C function
    );
}

void fault_dump(uint32_t* frame) {
    // frame[0]=r0, [1]=r1, [2]=r2, [3]=r3, [4]=r12, [5]=LR, [6]=PC, [7]=xPSR
    uint32_t fault_pc = frame[6];  // THIS is the instruction that caused the fault
    uint32_t cfsr = SCB->CFSR;     // reads the fault
    // Log to flash, blink LED, halt
}
```

---

2. Interrupt Hardware (NVIC)

How an interrupt works — CPU register level

Normal execution: CPU fetches instructions at addresses pointed to by PC PC: 0x1000 → 0x1004 → 0x1008 → 0x100C → ...

Interrupt fires (e.g., UART byte received):

- Peripheral sets flag bit in its status register

- NVIC (Nested Vectored Interrupt Controller) sees the flag
- NVIC forces CPU to:
    - Push registers (r0-r3, r12, LR, PC, xPSR) onto stack ← context save
    - Load ISR PC from VECTOR TABLE at fixed address
- Execute the ISR interrupt service routine function

```
CPU registers BEFORE interrupt:
┌──────────┬──────────┬────────┬──────────────┐
│ PC=0x100C│ SP=0x2000│ r0=42  │ LR=0x0FF8    │
└──────────┴──────────┴────────┴──────────────┘

Stack AFTER hardware auto-push (~12 cycles, zero software):
SP → ┌──────────────┐ 0x1FE0
     │ xPSR         │
     │ PC=0x100C    │  ← return address (resume here after ISR)
     │ LR=0x0FF8    │
     │ r12          │
     │ r3, r2, r1   │
     │ r0=42        │
     └──────────────┘ 0x2000 (original SP)

CPU registers DURING ISR:
┌──────────────┬──────────┬─────────────────┐
│ PC=ISR_addr  │ SP=0x1FE0│ LR=0xFFFFFFF9   │
└──────────────┴──────────┴─────────────────┘

ISR returns (bx lr with magic value):
  → CPU pops saved registers from stack and restore the values
  → PC restored to 0x100C → main code resumes exactly where it left off
```

Vector table — the hardware function pointer array

```
fixed Address 0x00000000 (flash start on Cortex-M):
┌──────────────────────────────────┐
│ 0x00: Initial SP value           │  ← loaded into SP at reset
│ 0x04: Reset_Handler address      │  ← first instruction after power-on
│ 0x08: NMI_Handler address        │
│ 0x0C: HardFault_Handler address  │
│ ...                              │
│ 0x6C: USART1_IRQHandler address  │  ← your ISR function pointer
│ 0x70: SPI1_IRQHandler address    │
│ 0x74: TIM2_IRQHandler address    │
└──────────────────────────────────┘
```

Priority Grouping for Nesting ISR

8 priority bits split into: [preemption priority | sub-priority]

- Preemption priority: higher prio ISR can interrupt lower prio ISR

- Sub-priority: tie-breaker when two ISRs pend simultaneously

Tail-chaining and Late-arriving

- w/o tail-chaining, back-to-back ISRs pay full pop then push overhead each switch.
- Tail-chaining skips that overhead when one ISR finishes and another is already pending (A -> B).
- Late-arriving lets a newly arrived higher-priority ISR preempt entry of a lower-priority one (B first, then A).

Callback — the software equivalent

Pattern : Interrupt -> ISR set flag -> Callback -> callback fn action on flag Everything in embedded is a **function pointer** — the difference is who calls it and when:

```
Vector table entry     = function pointer invoked by HARDWARE (CPU/NVIC)
Callback               = function pointer invoked by SOFTWARE (your loop/dispatcher)
vtable entry           = function pointer invoked by SOFTWARE (organized per class)
Task function pointer  = function pointer invoked by SCHEDULER
```

```
// Callback = fn pointer stored for runtime invocation
//       handles heavy interrupt tasks for light-weight ISR that doesnot
using Callback = void(*)(float);

Callback on_temp_ready = nullptr;
// handler not determined in compile time
void register_callback(Callback cb) { on_temp_ready = cb; }

void on_temp(float temp) {
  // user-defined handler, e.g. filter/log/control
}

void sensor_poll() {
    float temp = read_adc();
    if (on_temp_ready)       // check not null
        on_temp_ready(temp); // indirect call — one memory read + branch
}

int main() {
  register_callback(on_temp);  // register before first poll
  while (1) {
    sensor_poll();
  }
}
```

Hardware interrupt vs software callback

- **Who calls?**
    - Hardware interrupt: CPU hardware (NVIC)

    - Software callback: your code (loop/dispatcher)
- **Address stored where?**
    - Hardware interrupt: vector table in flash
    - Software callback: function pointer in RAM/struct
- **When called?**
    - Hardware interrupt: hardware event (pin, timer, peripheral)
    - Software callback: software condition (message received, timer elapsed)
- **Context save?**
    - Hardware interrupt: automatic by CPU (push registers to stack)
    - Software callback: normal function call (compiler manages stack frame)
- **Can preempt?**
    - Hardware interrupt: yes, stops whatever CPU was doing
    - Software callback: no, waits until caller invokes it

---

3. Communication Buses

UART / RS-232 / RS-485

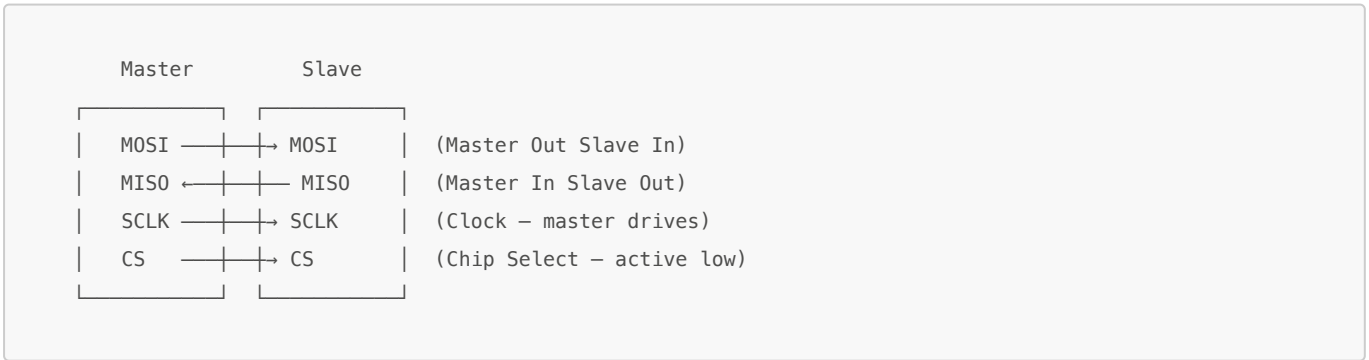
- UART = Universal Asynchronous Receiver/Transmitter
    - USART = Universal Synchrounous / Asynchronous Receiver/Transmitter
- RX / TX = Receiver pin / Transmitter pin

TX ———————————— RX & RX ———————————— TX

- w/o clock wire both side agrees on
    - baud rate, frame format, voltage level
    - Frame: [START][D0][D1]...[D7][PARITY?][STOP]
    - voltage: TTL/RS-232/RS-485 transceiver level

SPI (Serial Peripheral Interface)

Master-slave, synchronous, full-duplex



- Master drives clock → no baud rate agreement needed
- Full duplex: sends and receives simultaneously by shift register
- No addressing: CS pin selects which slave is active
- Speeds: typically 1-50 MHz (much faster than I2C/UART)
- Multiple slaves: one CS pin per slave, only one active at a time

Semi use: ADC reading (16-bit at 1 MHz = 16µs per sample), DAC output, FPGA communication, flash memory (recipe/log storage)

I2C (Inter-Integrated Circuit)

```
Multi-device bus, 2 wires with pull-up resistors:

  VCC──┐──────────┐
       [R]        [R]   (pull-ups: 4.7kΩ typical)
        |          |
  SDA ──┴──┬──┬──┬──┴──  (data — bidirectional, open-drain)
  SCL ─────┬──┬──┬─────  (clock — master drives)
           |  |  |
         [M] [S1][S2]  (master + multiple slaves on same 2 wires)
```

- Synchronous by a shared clock
- Address-based Each slave has a 7-bit address (set by hardware pins or fixed)
- Master initiates all communication so slave can't talk unprompted
- ACK/NACK after every byte
- Bus speed: 100 kHz (standard), 400 kHz (fast), 1 MHz (fast+)

Semi use: Temperature sensors (LM75, TMP117), EEPROMs (AT24C), humidity sensors, small displays (OLED)

CAN (Controller Area Network)

differential pair for robustness hardware arbitration:

```
CAN_H ──────┬──────┬──────┬──────┐
CAN_L ──────┬──────┬──────┬──────┐
            |      |      |      |
       [Node A][Node B][Node C][120Ω]  ← termination resistors at both ends
```

- Message-based, nodes put message to broadcast bus and interested node will pick
- Priority: lowest message ID wins bus arbitration (bit-dominant = 0)
    - ID 0x100 beats ID 0x200 automatically, no collision

CAN message frame:

```
┌─────┬────────────┬─────┬──────────────────────┬─────┬─────┬───┐
| SOF | ID (11b) | RTR | Data (0–8 bytes)   | CRC | ACK |EOF|
└─────┴────────────┴─────┴──────────────────────┴─────┴─────┴───┘
```

- SOF Start of Frame (dominant bit)
- ID Message identifier (11-bit standard, 29-bit extended)
- RTR Remote Transmission Request (request data from another node)
- CRC 15-bit CRC computed by hardware
- ACK All receivers acknowledge (pull bus dominant)

Semi use: Multi-axis motor controllers, gas panel valve sequencing, smaller tools without Ethernet

EtherCAT (Ethernet for Control Automation Technology)

Daisy-chain topology, deterministic, processing on the fly

```
Master → [Slave 1] → [Slave 2] → [Slave 3] → ... → back to Master
            ↕             ↕             ↕
         [Drive]       [I/O]        [Sensor]
```

- Standard Ethernet frame from master passes through each slave
- Timing for slaves are predictable -> good for control
- IN contrast star style switch introduce jitter time

Semi use: ASML wafer stage (synchronized multi-axis motion at nm precision), high-speed motion control, real-time I/O

USB (Universal Serial Bus)

```
Host-device centric topology (NOT peer-to-peer):

    Host (PC / Equipment Controller)
       |
    ┌──┴──┐
    | Hub | (optional)
    ├─────┤
    |     |
  [Dev1] [Dev2]
```

- Endpoint types: Control (setup), Bulk (data transfer), Interrupt (periodic), Isochronous (streaming)
- Requires USB stack software (dozens of KB) — rarely bare-metal on small MCU

Semi use: Equipment Controller ↔ PC data transfer, firmware update via DFU, virtual COM port for debug

Analog I/O (ADC / DAC)

ADC (Analog to Digital Converter) <-> DAC (Digital to Analog Converter)

Bus Comparison Table

UART is simple and cheap for local MCU-to-device links. RS-485 is UART-like framing on a stronger differential physical layer for longer/noisier runs. SPI is fast and deterministic for short board-level links as no-addressing I2C is low-pin-count and good for many low-speed peripherals. CAN is robust multi-node messaging with built-in arbitration and fault handling. EtherCAT is deterministic industrial Ethernet for synchronized motion/control. USB is host-centric and high throughput for peripherals.

---

4. System Layer — No-OS vs RTOS vs Full OS

Bare Metal (No-OS)

```
main() {
    init_hardware();
    while (1) {
        task_a();  // always runs, no OS, no scheduler
        task_b();  // just a function call
        task_c();
    }
}

Memory footprint: 0 bytes for "OS" — all RAM available for your code
Context switch: none (no task stacks)
Scheduling: implicit (order of function calls in loop)
Interrupt: hardware ISR → set flag → main loop checks flag
```

Variants:

Pattern	How it works	Limitation
Super-loop	while(1) { do everything }	Fastest task dictates cycle time
ISR + flags	ISR sets volatile flag, main loop polls	Still no preemption
Cooperative scheduler	Timer tick + task table with periods	Tasks must not block

Semi use: Individual sensor MCUs — thermocouple ADC, pressure gauge, single-axis motor. Response time <10µs, single responsibility.

RTOS (Real-Time Operating System)

```
┌──────────────────────────────────────────┐
│ RTOS Kernel                              │
├──────────┬──────────┬──────────┬─────────┤
│ Task A   │ Task B   │ Task C   │ Idle    │
│ Stack A  │ Stack B  │ Stack C  │ Stack   │
│ Prio: 3  │ Prio: 2  │ Prio: 1  │ Prio: 0 │
└──────────┴──────────┴──────────┴─────────┘
```

Kernel tick ISR (SysTick, every 1ms):

1. Check if higher-priority task is now ready
2. If yes → context switch:
    - Save current task registers to its stack (PSP)
    - Load new task registers from its stack (PSP)
    - Return from ISR into new task
3. If no → return to same task

Context switch cost: ~10µs on Cortex-M4 at 168 MHz (save/restore ~32 registers)

Synchronization primitives:

Primitive	Purpose	Typical use
Task	Independent thread with own stack + priority	Each subsystem = 1 task
Mutex	Mutual lock exclusion (one owner at a time)	Protect shared hardware (SPI bus)
Semaphore	Counting signal (producer/consumer)	ISR -> task "data ready"
Event flags	Bitfield of conditions (wait for any/all)	Multiple ready signals
Timer	Software timer (calls callback after delay)	Timeout, periodic actions

RTOS vendors in semiconductor:

- VxWorks
  - License: Commercial ($$$)
  - Used by: Applied Materials, TEL, ASM
  - Strengths: Certified (DO-178C), mature
- QNX
  - License: Commercial
  - Used by: Lam Research, ASM
  - Strengths: Microkernel, fault isolation
- FreeRTOS
  - License: MIT (free)
  - Used by: New designs, prototypes
  - Strengths: Simple, huge community
- Zephyr
  - License: Apache 2.0
  - Used by: Emerging
  - Strengths: Modern, device tree, upstream drivers
- RT-Linux
  - License: GPL
  - Used by: ASML, ASM, KLA
  - Strengths: Linux ecosystem + RT patch

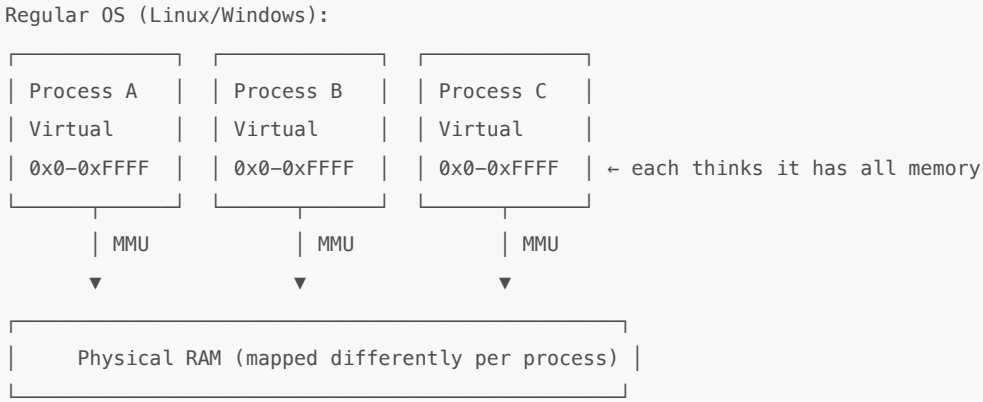
Semi use: Chamber controller — manages heater PID, pressure control, gas sequencing, safety interlocks, host communication as separate tasks with priorities.

RTOS Tasks vs OS Processes

- **Address space**
  - RTOS Task: **Shared** — all tasks see all RAM (flat memory, no MMU)
  - OS Process: **Isolated** — each process has its own virtual address space (MMU enforced)
- **Protection**
  - RTOS Task: None by default (optional MPU = coarse, 8 regions)
  - OS Process: Full (MMU = fine-grained page-level isolation)
- **Stack**

    - RTOS Task: PSP per task, but heap/globals are shared
    - OS Process: Completely separate stack, heap, globals
- **Context switch cost**
    - RTOS Task: ~10µs (save/restore registers only)
    - OS Process: ~50-100µs (flush TLB, switch page tables, cache effects)
- **Creation cost**
    - RTOS Task: Statically allocated at boot (no `fork`)
    - OS Process: `fork()`/`exec()` — copies page tables, allocates PID
- **Crash isolation**
    - RTOS Task: Task corrupts shared RAM -> all tasks affected
    - OS Process: Process segfaults -> only that process dies
- **Scheduling**
    - RTOS Task: **Deterministic** — highest priority always runs, guaranteed within us
    - OS Process: **Best-effort** — CFS/priority can be preempted by kernel, no hard guarantees
- **System calls**
    - RTOS Task: Direct function call (same privilege level, no mode switch)
    - OS Process: Trap to kernel (user->kernel mode switch, ~1us overhead)
- **Memory footprint**
    - RTOS Task: ~512B-4KB per task (just the stack)
    - OS Process: ~4MB+ virtual space per process (page tables, heap, libs)

The key difference — no MMU on Cortex-M:

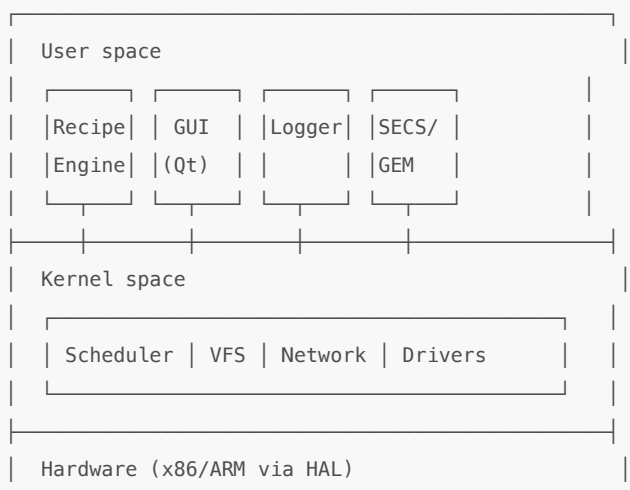


Why not just use Linux on everything?

Requirement	Linux process	RTOS task
Respond within 10µs guaranteed	Cannot guarantee	Yes
Run on 64KB RAM MCU	Impossible (needs ~8MB+)	Yes
Deterministic worst-case timing	No (kernel can delay)	Yes (priority preemption is immediate)
Cost per unit (BOM)	$5+ SoC with MMU	$2 Cortex-M with no MMU

If you use **RT-Linux** (PREEMPT_RT) on a Cortex-A with MMU, you get processes with isolation AND ~50µs real-time. That's what ASML uses for upper layers. But on a $2 Cortex-M chip controlling a heater PID loop at 10kHz — tasks are just function pointers with separate stacks, scheduled by a tiny kernel (~5KB).

Full OS (Linux / Windows)



- Virtual memory (MMU): each process thinks it has all RAM
- Preemptive multitasking: kernel can interrupt any process
- File systems: ext4, NTFS — recipe storage, data logging
- Network stack: full TCP/IP, HTTP, SSH
- Not real-time by default (scheduling latency: 1–10ms)
  - PREEMPT_RT patch: reduces worst case to ~50µs (still not as good as RTOS)

Semi use: Equipment Controller — recipe engine, operator GUI, SECS/GEM factory communication, data collection. Connects DOWN to RTOS via Ethernet/PCIe/shared memory.

Decision Matrix

Criteria	No-OS	RTOS	Full OS
Tasks	1-3	4-20+	Unlimited
Deadline	Hardest (<10µs)	Hard (100µs-10ms)	Soft (>10ms)
RAM	<64KB	64KB-1MB	>1MB
Flash	<256KB	256KB-2MB	>4MB

Criteria	No-OS	RTOS	Full OS
Needs file system?	No	Maybe (FatFS)	Yes
Needs TCP/IP?	No	Lightweight (lwIP)	Full stack
Needs GUI?	No	No	Yes
Code complexity	Low	Medium	High
Determinism	Highest	High	Low (without RT patch)
Failure isolation	None (one crash = all crash)	Per-task (MPU)	Per-process (MMU)
Semi example	Thermocouple MCU	Chamber RTOS	Equipment PC

Boot Sequence (what happens before main)

```
Power on / Reset:
  1. CPU loads SP from vector table[0] (0x00000000)
  2. CPU loads PC from vector table[1] (0x00000004) → Reset_Handler

Reset_Handler (startup assembly):
  3. Copy .data section from Flash → RAM (initialized globals)
  4. Zero .bss section in RAM (uninitialized globals)
  5. Call SystemInit() → configure clock tree (HSE → PLL → 168 MHz)
  6. Call __libc_init_array() → C++ static constructors
  7. Call main()

In main():
  8. hal::init() → configure all peripherals (GPIO, UART, SPI, timers)
  9. Start scheduler (RTOS) or enter super-loop (bare metal)
```

Linker Script (memory map)

```
Linker script (.ld file) tells the linker WHERE to put things:

MEMORY {
    FLASH (rx)  : ORIGIN = 0x08000000, LENGTH = 1024K
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}

SECTIONS {
    .isr_vector : { *(.isr_vector) } > FLASH     /* vector table at flash start */
    .text       : { *(.text*) } > FLASH           /* code */
    .rodata     : { *(.rodata*) } > FLASH         /* const data, lookup tables */
    .data       : { *(.data*) } > RAM AT> FLASH   /* initialized globals (copied at boot) */
    .bss        : { *(.bss*) } > RAM              /* uninitialized globals (zeroed at boot) */
    ._stack     : { . = . + 0x2000; } > RAM       /* 8KB stack */
}

Memory map result:
```

```
┌──────────────────┐ 0x08100000
│                  │
│  FLASH (1MB)     │  .isr_vector → .text → .rodata → .data (init values)
│                  │
├──────────────────┤ 0x08000000
│                  │
│  (gap)           │
│                  │
├──────────────────┤ 0x20020000
│  Stack (top)     │  ← SP starts here, grows DOWN
│  ...             │
│  Heap            │  ← grows UP (if malloc used)
│  .bss            │  ← zeroed at boot
│  .data           │  ← copied from flash at boot
├──────────────────┤ 0x20000000
```

Watchdog Timer

```
Independent Watchdog (IWDG):
  - Separate clock (LSI, ~32 kHz) — runs even if main clock fails
  - Must be "kicked" periodically or system resets
  - Once started, CANNOT be stopped (by design — prevents software from disabling safety)

Window Watchdog (WWDG):
  - Must be kicked WITHIN a time window (not too early, not too late)
  - Detects both stuck code (too late) and runaway code (too fast)

             ┌─── too early (reset) ──┐
             │                        │
  ──────────[═══════ WINDOW ═══════]──────────→ too late (reset)
             │                        │
             └─── KICK HERE ──────────┘

Semi safety: if control loop hangs → watchdog fires → hardware reset
  → startup code checks reset reason → enters safe state (valves close, heaters off)
  → logs fault for diagnosis
```

Firmware Update

```
Bootloader (lives in protected flash region, never overwritten):

Boot:
  1. Check "update pending" flag (set by application before reset)
  2. If set: receive new firmware over UART/USB/CAN
  3. Verify CRC / digital signature
  4. Write to application flash region
  5. Clear flag, reset → run new firmware

Dual-bank flash (more robust):
  ┌──────────┬──────────┐
```

```
    | Bank A   | Bank B   |
    | (active) | (update) |
    └──────────┴──────────┘


  1. Running from Bank A
  2. Write new firmware to Bank B (application continues running!)
  3. Verify Bank B CRC
  4. Swap bank mapping (one register write)
  5. Reset → now running from Bank B
  6. If Bank B crashes → watchdog → bootloader → swap back to Bank A (rollback)

Semi use: field firmware update without opening tool, automatic rollback on failure
```

---

5. Semi Equipment Architecture (4-Layer Stack)

- Factory MES (Manufacturing Execution System)
  - SECS/GEM protocol over TCP/HSMS
- Equipment Controller (Windows/Linux, Full OS)
  - Recipe engine, GUI, data collection, SECS/GEM driver
  - Communication: Ethernet TCP/UDP, PCIe, reflective memory
- Real-Time Controller (VxWorks/QNX/RT-Linux, RTOS)
  - Chamber control, task scheduling, safety interlocks
  - Communication: SPI, I2C, UART, CAN to sensors below
- Sensor/Actuator MCUs (Bare metal, No-OS)
  - Single-function: ADC→SPI, PID→PWM, I2C sensor read
  - Communication: SPI/I2C/UART up to RTOS controller

Bus usage per layer boundary:

Boundary	Buses used	Why
Factory → Equipment	Ethernet (TCP/HSMS)	Standard factory protocol, long distance
Equipment → RTOS	Ethernet, PCIe, reflective memory	High bandwidth, medium latency OK
RTOS → Sensor MCU	SPI, I2C, UART, CAN	Low pin count, short distance, simple
MCU → Physical sensor	Analog (ADC), SPI (digital sensor)	Direct electrical connection